



Nami Challenge

Montar el entorno del desafío

Primero, debemos iniciar el entorno para completar el desafío para ello es necesario levantar el contenedor de docker utilizando el primer comando documentado en  [Utils](#).

Luego, debemos cambiar de usuario al correspondiente al reto, para ello utilizamos el segundo comando documentado en  [Utils](#).

```
PS C:\Users\Usuario\Documents\GitHub\data-encryption\ctf_onepice_symmetric_cipher> docker exec -it nami_challenge bash
nobody@d2e6fc7f2645:/home/nami$ su nami
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

nami@d2e6fc7f2645:~$
```

La contraseña es la bandera del  [Usopp Challenge](#).

Extraer bandera de **flag.txt**

El primer paso para extraer la bandera del desafío, es encontrar todos los posibles archivos que puedan contenerla, para ello, utilizamos el primer comando documentado en  [Utils](#).

```
nami@d2e6fc7f2645:~/ONEPIECE$ find . -type f -iname "*flag*"
./East_Blue/Baratie/Casa_de_Zeff/flag.txt
./East_Blue/Romance_Dawn/Casa_de_Shanks/flag.txt
./East_Blue/Romance_Dawn/Casa_de_Makino/flag.txt
./Alabasta/Alubarna/Casa_de_Vivi/flag.txt
./Alabasta/Yuba/Casa_de_Toto/flag.txt
./Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom/flag.txt
./Whole_Cake_Island/Cacao_Island/Casa_de_Big_Mom/flag.txt
./Water_7/Blue_Station/Casa_de_Paulie/flag.txt
./Water_7/Carpenters_Cafe/Casa_de_Paulie/flag.txt
./Pirate_Island/Pirates_Ship/Casa_de_Gold_Roger/flag.txt
nami@d2e6fc7f2645:~/ONEPIECE$
```

Luego de obtener todos los potenciales archivos, verificamos el contenido de cada uno, utilizando el comando `cat`, hasta encontrar el archivo que contiene texto cifrado.

```
nami@d2e6fc7f2645:~/ONEPIECE$ cat ./East_Blue/Romance_Dawn/Casa_de_Shanks/flag.txt
6f64db0ea6cdd20966bb469662c6b520f64501fa10362b49e6cc02e2fe4f9e8004dbc69f6dnami@d2e6fc7f2645:~/ONEPIECE$
nami@d2e6fc7f2645:~/ONEPIECE$
```

Para descifrar la bandera, utilizamos un script en Python que implementa el algoritmo de cifrado ChaCha20 para descifrar el texto cifrado, utilizando una clave y un nonce derivados de mi carnet de estudiante como identificador de usuario.

```
#!/usr/bin/env python3

import sys
import binascii
from Crypto.Cipher import ChaCha20

def generate_key_nonce(user_id):
    """Genera la clave y el nonce a partir del ID de usuario"""
    key = (user_id.encode() * 32)[:32] # Derivar clave de 256 bits del ID
    nonce = (user_id.encode() * 8)[:8] # Derivar nonce de 64 bits del ID
    return key, nonce
```

```

def chacha20_decrypt(ciphertext, user_id):
    """Descifra el texto usando ChaCha20"""
    key, nonce = generate_key_nonce(user_id=user_id)
    cipher = ChaCha20.new(key=key, nonce=nonce)
    plaintext = cipher.decrypt(ciphertext)
    return plaintext

def main():
    """Función principal del script"""
    if len(sys.argv) != 3:
        print(f"Uso: {sys.argv[0]} <ruta_del_archivo> <user_id>")
        sys.exit(1)

    file_path = sys.argv[1]
    user_id = sys.argv[2]

    try:
        # Leer el contenido del archivo
        with open(file_path, 'r') as file:
            hex_content = file.read().strip()

        # Verificar que el contenido sea hexadecimal válido
        try:
            ciphertext = binascii.unhexlify(hex_content)
        except binascii.Error:
            print(f"Error: El contenido del archivo no es un hexadecimal válido")
            sys.exit(1)

        # Descifrar el contenido
        plaintext = chacha20_decrypt(ciphertext, user_id)

        # Intentar mostrar como texto
        try:
            decoded = plaintext.decode('utf-8')
            print(f"Texto descifrado: {decoded}")
        except UnicodeDecodeError:
            print(f"El texto descifrado no es UTF-8 válido.")
            print(f"Bytes descifrados: {plaintext}")

```

```

        print(f"Representación ASCII: {''.join(chr(b) if 32 <= b < 127 else '.' for b in b_list)")

    except FileNotFoundError:
        print(f"Error: El archivo {file_path} no existe")
        sys.exit(1)
    except Exception as e:
        print(f"Error: {str(e)}")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).

```

nami@d2e6fc7f2645:~/ONEPIECE$ nano decrypt.py
nami@d2e6fc7f2645:~/ONEPIECE$ chmod +x decrypt.py
nami@d2e6fc7f2645:~/ONEPIECE$ ./decrypt.py ./East_Blue/Romance_Dawn/Casa_de_Shanks/flag.txt "21542"
Texto descifrado: FLAG_974f3f8886806d53a98ad74b1df57cea
nami@d2e6fc7f2645:~/ONEPIECE$

```

Obteniendo así, la bandera descriptada.

```
FLAG_974f3f8886806d53a98ad74b1df57cea
```

Extraer párrafo de **poneglyph.jpeg**

Al igual que con la extracción de la bandera, el primer paso es encontrar la carpeta que contiene la imagen cifrada, para ello, utilizamos el cuarto comando documentado en  [Utils](#).

```
nami@d2e6fc7f2645:~/ONEPIECE$ find . -type f -name "*poneglyph*"
./Water_7/GalleyLa_Headquarters/Casa_de_Paulie/poneglyph.zip
nami@d2e6fc7f2645:~/ONEPIECE$ cd ./Water_7/GalleyLa_Headquarters/Casa_de_Paulie/
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie$ ls
poneglyph.zip
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie$
```

Luego de obtener la carpeta, nos dirigimos hacia su directorio y la extraemos su contenido utilizando el quinto comando documentado en  [Utils](#). La contraseña a utilizar es la misma que la del usuario.

```
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie$ 7z x poneglyph.zip -oponeglyph

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=C,Utf16=off,HugeFiles=on,64 bits,8 CPUs Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz (806EA),ASM,AES-NI)

Scanning the drive for archives:
1 file, 53013 bytes (52 KiB)

Extracting archive: poneglyph.zip
--
Path = poneglyph.zip
Type = zip
Physical Size = 53013

Enter password (will not be echoed):
Everything is Ok

Size:      53045
Compressed: 53013
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie$
```

Para poder utilizar el comando de extracción, es posible que sea necesario instalar ciertas librerías. Para ello, utilice el noveno comando documentado en  [Utils](#).

Para descifrar el parrafo, utilizamos un script que:

1. Extrae el texto de la imagen.
2. Toma el texto de la imagen y realiza una operación XOR entre el texto cifrado y mi carnet de estudiante (21542).

```
#!/bin/bash
```

```
#Verificar que se proporcione el carné como argumento
if [ $# -ne 1 ]; then
    echo "Uso: $0 <tu_carné>"
```

```

    exit 1
fi

STUDENT_ID=$1

#Definir los nombres de los directorios y archivos
IMAGE_PATH="poneglyph.jpeg" # Asumiendo que está en el mismo directorio
OUTPUT_FILE="decrypted.txt" # Archivo donde se guardará el texto descifrado

#Crear el archivo luffy_xor.py con el contenido correcto
cat > luffy_xor.py << 'EOF'
def xor_cipher(text, key):
    # Convertir tanto el texto como la clave a bytes
    if type(text) == str:
        text_bytes = text.encode()
    else:
        text_bytes = text
    key_bytes = key.encode() # Convertir la clave a bytes

    # Cifrar con XOR
    cipher_text = bytes([text_bytes[i] ^ key_bytes[i % len(key_bytes)] for i in range(len(text_bytes))])

    return cipher_text
EOF

#Crear un script Python temporal para la extracción
cat > extract_script.py << 'EOF'
from PIL import Image
import piexif
import sys
from luffy_xor import xor_cipher

def extraer_texto_metadata(imagen_path):
    # Abrir la imagen
    img = Image.open(imagen_path)

    # Obtener los metadatos EXIF
    try:

```

```

exif_data = img.info.get('exif')
if not exif_data:
    return None

exif_dict = piexif.load(exif_data)

# Obtener el texto almacenado en 'Artist'
texto = exif_dict['0th'].get(piexif.ImageIFD.Artist)
if texto:
    return texto
return None
except Exception as e:
    print(f"Error al extraer metadatos: {e}")
    return None

# Obtener argumentos
if len(sys.argv) != 4:
    print("Uso: python script.py <imagen_path> <carne> <output_file>")
    sys.exit(1)

imagen_path = sys.argv[1]
student_id = sys.argv[2]
output_file = sys.argv[3]

# Extraer y descifrar
texto_cifrado = extraer_texto_metadata(imagen_path)
if texto_cifrado:
    try:
        # Aplicar XOR y decodificar si es posible
        texto_descifrado = xor_cipher(texto_cifrado, student_id)
        try:
            # Intentar decodificar como UTF-8
            decoded_text = texto_descifrado.decode('utf-8')

            # Guardar en archivo
            with open(output_file, 'w') as f:
                f.write(decoded_text)

```

```

        # Mostrar en pantalla
        print(decoded_text)

except UnicodeDecodeError:
    # Si falla la decodificación UTF-8, guardar como bytes en archivo bina
    with open(output_file, 'wb') as f:
        f.write(texto_descifrado)

    # Mostrar en pantalla
    print(texto_descifrado)
    print(f"\nEl contenido binario descifrado se ha guardado en: {output_fil

except Exception as e:
    print(f"Error al descifrar: {e}")
else:
    print("No se encontraron metadatos EXIF o no hay campo Artist en la image
EOF

#Ejecutar el script en el mismo contenedor Docker
pip install pillow piexif

python3 extract_script.py "$IMAGE_PATH" "$STUDENT_ID" "$OUTPUT_FILE"

echo "Texto descifrado guardado en: $OUTPUT_FILE"

rm -f luffy_xor.py extract_script.py

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).


```

nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie/poneglyph$ nano decryptImg.sh
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie/poneglyph$ chmod +x decryptImg.sh
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie/poneglyph$ ./decryptImg.sh "21542"
Defaulting to user installation because normal site-packages is not writeable
Collecting pillow
  Downloading pillow-11.2.1-cp310-cp310-manylinux_2_28_x86_64.whl (4.6 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.6/4.6 MB 4.3 MB/s eta 0:00:00
Collecting piexif
  Downloading piexif-1.1.3-py2.py3-none-any.whl (20 kB)
Installing collected packages: pillow, piexif
Successfully installed piexif-1.1.3 pillow-11.2.1
before telling Robin to search out the Poneglyphs and draw her own conclusions regarding the lost history.
Texto descifrado guardado en: decrypted.txt
nami@d2e6fc7f2645:~/ONEPIECE/Water_7/GalleyLa_Headquarters/Casa_de_Paulie/poneglyph$ █

```

Obteniendo así, el párrafo descriptado.

before telling Robin to search out the Poneglyphs and draw her own conclusions